



DURBAN UNIVERSITY OF TECHNOLOGY
INYUVESI YASETHEKWINI YEZOBUCHWEPHESHE

FACULTY OF ACCOUNTING AND INFORMATICS

DEPARTMENT OF INFORMATION TECHNOLOGY

2025

FEBRUARY SPECIAL EXAMINATION

MARKING MEMORANDUM

INSTRUCTIONAL PROGRAMME : B INFORMATION & COMMUNICATION TECHNOLOGY

INSTRUCTIONAL OFFERING : INTEGRATIVE PROGRAMMING AND TECHNOLOGIES III

PAPER NUMBER : 1

SUBJECT CODE/S : IPRT301

DATE : 27 FEBRUARY 2025

DURATION : 3 HOURS

TIME : 09H00 - 12H00

TOTAL MARKS : 100

NUMBER OF PAGES : 9 (INCLUDING COVER PAGE)

EXAMINER/S : BOAMAH-ABU, CHARLES (PhD)

MODERATOR/S : MR RUSHIL RAGHAVJEE

INSTRUCTIONS/REQUIREMENTS:-

1. This examination is closed book
2. Answer all questions
3. Ensure your handwriting is readable

Do not turn the page until permission is given

Question 1

10 marks

In each of the following, state whether the statement is **true** or **false**. Write **T** (for **true**) or **F** (for **false**) for your answer.

1. In general, cohesion among members of a class is preferable.
2. Java allows a subclass to inherit two or more superclasses.
3. An abstract class can extend another abstract class.
4. An instance of a subclass can behave as the superclass.
5. An appropriate method for an instance of a subclass is selected at run time.
6. In general, members of an interface are declared private.
7. An interface cannot contain constants.
8. A superclass cannot contain method implementation.
9. An interface cannot contain method implementation.
10. Method override is when two or more methods in the same class have the same name but different parameter list.

Answers: 1. T 2. F 3. T 4. T 5. T 6. F 7. F 8. F 9. T 10. F

Question 2

12 marks

Briefly answer the following.

1. Explain why you will choose JSON over XML for data exchange. (2 marks)
2. Explain how integrative programming supports scalability. (2 marks)
3. Mention and briefly describe **one** challenge with integrative programming. (2 marks)
4. Explain why design patterns are important in integrative programming? (2 marks)
5. Describe microservices architecture. Mention one benefit. (4 marks)

Answers

1. Any correct answer, e.g., JSON code is smaller and faster to execute than XML✓✓
2. Any correct answer, e.g., integrative programming combines different components (e.g., applications). This allows new components to be added to the existing components✓✓
3. Any correct answer: integration of many and different components introduces complexity (e.g., security, management etc.) ✓✓
4. Any correct answer, e.g., provides guidelines for designing solutions ✓✓
5. Any correct answer, e.g., Microservice consists of small components, each with its own processor, and performs a specific function✓✓. Any benefit (, e.g., fault tolerance, scalability, loose coupling; etc) ✓✓

Question 3

6 marks

Briefly describe when the following design patterns are useful (example: Factory- when you need to create instances of subclasses of a superclass).

1. Singleton (2 marks)
2. Adaptive Adapter (2 marks)
3. DAO (2 marks)

Answer

	Design pattern	Used to...
1	Singleton	When you need to create only one instance of a class✓✓
2	Adapter	When you need different classes to work together✓✓
3	DAO	When you need to separate business objects from operations✓✓

Question 4

14 marks

I is an interface, A is an abstract and C is a concrete class. C inherits A; and A implements I.

1. Complete the class header for A below. (2 marks)

```
public _____ {  
  
}
```

```
public abstract class A implements I ✓✓
```

2. Complete the class header for C below. (2 marks)

```
public _____ {  
  
}
```

```
public class C extends A ✓✓{
```

3. In each of the following, state whether the statement is valid or invalid about I, A and C.

Explain your answer.

(10 marks)

- a. I i1=new I();
- b. I i2=new A();
- c. I i3=new C();
- d. C c1=new A();
- e. A a1=new C();

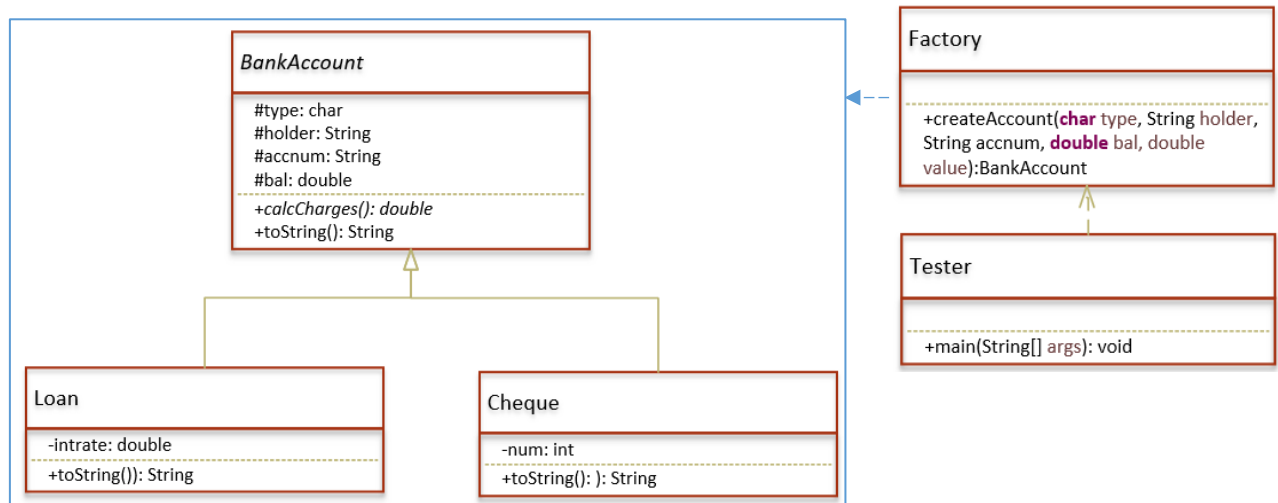
Any correct answer

- a. I i1=new I(); invalid; cannot create an instance of an interface✓✓
- b. I i2=new A(); invalid; cannot create an instance of an abstract class✓✓
- c. I i3=new C(); valid; can create an instance of a concrete class and assign to a variable of an interface the superclass "inherits" ✓✓
- d. C c1=new A(); invalid; cannot create an instance of a abstract class✓✓
- e. A a1=new C(); valid; can create an instance of a concrete class and assign to a variable of superclass it inherits✓✓

Question 5

22 marks

The UML diagram below shows the classes **BankAccount**, **Loan**, **Cheque**, **Factory** and **Tester**.



- BankAccount** is an abstract class and contains **type** (account type), **holder** (account holder), **accnum** (account number) and **bal** (current balance). **calcCharges()** is an abstract method.
- Loan** and **Cheque** inherit **BankAccount**.
- Loan** contains **intrate** (the interest rate **per annum**). The method **calcCharges()** returns the **monthly** charges (i.e., interest payable) on the loan
- Cheque** contains **num** (the number of transactions in a month). Each transaction is charged at R2.0. The method **calcCharges()** returns the monthly charges for transactions on the account
- Factory** is a factory class. The method **createAccount()** takes **type**, **holder**, **accnum**, **bal** and **value** (a decimal) as parameters. If **type** is:
 - 'L' or 'l': the method uses the parameters to create and return an instance of **Loan**
 - 'C' or 'c': the method uses the parameters to create and return an instance of **Cheque**
- Tester** creates an instance of **Factory** and calls its method to return an instance of a subclass of **BankAccount** (i.e., **Loan** or **Cheque**), which is then displayed on the screen.

Use the information above to answer 1 to 5 below.

- Write the method **calcCharges()** in the class **BankAccount**. (2 marks)

Answer: **public abstract double calcCharges();** ✓✓

- Write the header of the class **Loan**. (2 marks)

Answer **public class Loan extends BankAccount** ✓✓

- Write the method **calcCharges()** in the class **Loan**. (3 marks)

Answer: **public double calcCharges()** ✓ {
 return bal * intrate/12; ✓✓
 }

- Below, complete the method **createAccount(char type, String name, double num)** in the class **Factory**. Assume the relevant classes and their methods (including **constructors**) have been implemented. (10 marks)

```

        createAccount(char type, String
holder, String accnum, double bal, double value) {
    _____
    _____
    _____
    _____
    _____
}

```

Answer

```

public BankAccount createAccount(char type, String holder, String accnum, double bal, double
value) {
    BankAccount ba=null;
    ✓✓if(Character.toUpperCase(type)=='L') {
        ✓✓ba=new Loan(type, holder, accnum, bal, value);
    }
    ✓✓if(Character.toUpperCase(type)=='C') {
        //create instance of Loan
        ✓✓ba=new Cheque(type, holder, accnum, bal, (int)value);
    }
    ✓return ba;
}

```

5. Assume the relevant classes and their methods (including **constructors** and **toString()**) have been implemented. Below, complete **main()** in **Tester** as follows:
1. Create an instance of **Factory** called **f**. (1 mark)
 2. Use **f** to call an appropriate method (with the values 'l', "Joe May", "50-220-321-123", 10000, 0.12) to return the instance of a subclass. (3 marks)
 3. Display the instance of the subclass on the screen. (1 mark)

```

package bankfactory;
public class Tester {
    public static void main(String[] args) {
        _____
        _____
        _____
    }
}

```

```

public class Tester {
    public static void main(String[] args) {
        Factory f=new Factory();✓
        BankAccount b1=f.createAccount('l', "Joe May", "50-220-321-123", 10000, 0.12); ✓✓✓
        System.out.println(b1); ✓
    }
}

```

Question 6

11 marks

The table below shows the final marks for students in a module. Each student record consists of **student Id (studid)**, **dp** and **exam** mark.

StudentId	Dp	Exam
20200001	50	60
20200002	50	50
20200003	60	60

The XML code for the data is in a file (**marks.xml**). The **root** element is **Mark** and the child element **Student**.

- Complete the XML code segment below for the first record (i.e., for student 20200001). (5 marks)

```
<?xml version="1.0" encoding="_____"?>
<Mark>
    _____
    _____
    _____
    _____
    .....
    .....
</Mark>
```

Answer:

```
<?xml version="1.0" encoding="UTF-8"?>
<Mark>
    <Student>
        <studid>20200001</studid>
        <dp>50</dp>
        <exam>60</exam>
    </Student>
</Mark>
```

- Complete the code segment below for the DTD of the XML document (**marks.xml**). (6)

```
<?xml encoding="_____"?>
<!ELEMENT _____>
<!ELEMENT _____>
...
...
```

Answer:

```
<?xml encoding="UTF-8"?> (
<!ELEMENT Mark (Student)+>
```

Type text here

<!ELEMENT Student ✓ (studid,dp,exam) ✓ ✓ >
...
...

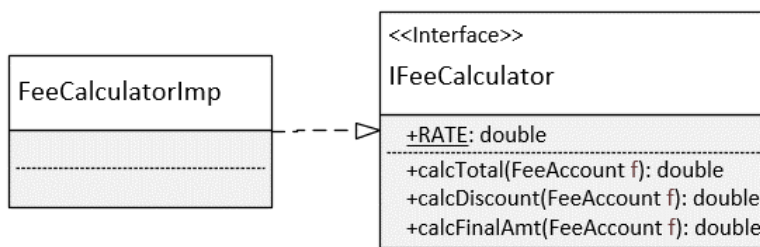
Question 7

25 marks

LFA International is a fee paying Grade 12 finishing school. The school offers a family discount: if there is more than one learner from a family, the first learner pays full fees and each of the rest receive 5% discount. For example, if there are three learners, the first pays full fees and the other two receive a discount of 5% each.

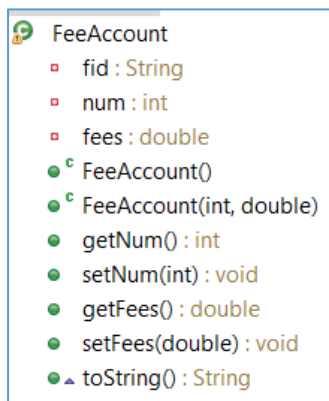
You are required to write an RMI application to calculate for each family, the total fees for all children, family discount and final amount payable (i.e., total fees less family discount).

The UML diagram below shows the remote interface and remote class.



- **RATE** is the discount rate, a constant equal to 5%
- **calcTotal(FeeAccount fPar)** takes a **FeeAccount** as parameter and returns the total fee for all children
- **calcDiscount(FeeAccount fPar)** takes a **FeeAccount** as parameter and returns the discount
- **calcFinalAmt (FeeAccount fPar)** takes a **FeeAccount** as parameter and returns the final amount payable

Below is the class **FeeAccount**. **fid** is a family's id; **num** is the number of learners in the school; **fees** is the school fees.



Use the information to answer 1 to 5 below.

1. Write the class header of **FeeAccount** for this distributed application. (2 marks)

```
public class FeeAccount implements Serializable ✓✓
```

2. Write the code for **IFeeCalculator**. (9 marks)

Answer

```
public interface IFeeCalculator✓ extends Remote✓{
    double RATE=0.05; ✓
    double calcTotal(FeeAccount f) throws Exception; ✓✓
    double calcDiscount(FeeAccount f) throws Exception; ✓✓
    double calcAmtDue(FeeAccount f) throws Exception; ✓✓
}
```

3. Write the class header for **FeeCalculatorImp**.

(3 marks)

Answer

```
public class FeeCalculatorImp✓ extends UnicastRemoteObject✓ implements IFeeCalculator✓
```

4. Write the method **calcFinalAmt()** in **FeeCalculatorImp** to call **calcTotal()** and **calcDiscount()** and return the final amount payable by a family.

(4 marks)

Answer

```
public double ✓ ½ calcFinalAmt(FeeAccount f) ✓ {
    return ✓ ½ calcTotal(fPar)-calcDiscount(f) ✓✓;
}
```

5. Complete the code segment below for a class **Server** which creates, registers and binds an instance of **FeeCalculatorImp** (called **fc**) with the reference "**fees**". Assume **FeeCalculatorImp** has been implemented. Use port **2222**.

(7 marks)

```
...
...
public class Server {
    public static void main(String[] args) _____{
        FeeCalculatorImp fc= _____;
        Registry r= _____;
        _____;
    }
}
```

Answer

```
...
...
public class Server {
    public static void main(String[] args) throws Exception✓{
        FeeCalculatorImp fc=new FeeCalculatorImp();✓
        Registry r=LocateRegistry.createRegistry(2222); ✓✓
        r.rebind✓ ("fees", fc); ✓✓
    }
}
```